

Lab5 Report: Value-Based Reinforcement Learning

張苙烜/ 資科工碩 313551079

1 Introduction

Lab5 練習 Reinforcement Learning 中的 Q-Learning，了解透過網路儲存 Q value 的實際訓練流程。Task1 針對經典的 CartPole 環境進行 Vanilla DQN 的實作，透過 MLP 架構來近似 Q 值，再根據 ϵ -greedy、target network 與 replay buffer 等機制，建立能穩定學習基本策略的模型。Task2 轉換為高維的 RGB 輸入 (Atari Pong-v5)，網路架構改成 CNN，並且需要用 AtariProcessor 處理 image，試圖讓 DQN 可以解決高維任務。Task3 探討並實作了三種常見的 DQN 改良技術：Double DQN、Prioritized Experience Replay (PER) 與 Multi-Step Return，實驗解決 DQN 中的 Overestimation、Low sample efficiency、Learning long-term rewards 的各類問題，並且比較 vanilla DQN 和 Enhanced DQN 的差別。

2 Implementation

2.1 Task1 CartPole Implement Details

建立一個 MLP hidden layers num = 2

```
1     if len(input) == 1: # For CartPole
2         layers = []
3         layers.append(nn.Linear(input[0], hidden_dim))
4         layers.append(nn.ReLU())
5         for _ in range(num_layers-1):
6             layers.append(nn.Linear(hidden_dim, hidden_dim))
7             layers.append(nn.ReLU())
8         layers.append(nn.Linear(hidden_dim, num_actions))
9         self.network = nn.Sequential(*layers)
```

在 init 中加入 memory 初始，用 Double-Ended Queue 儲存 memory_size 大小的 buffer

```
1     self.memory = deque(maxlen=args.memory_size)
```

當 replay_start_size 到達時，開始隨機從 buffer 中取樣，並且利用 ϵ -greedy 策略選擇 action，epsilon 隨著迭代下降，越高代表選擇越隨機探索，慢慢降低後趨近 greedy 每次都選 Q value 比較大的。然後在 train 裡面的 code 如下，利用 qnet 計算當前 state 和 action 下的 q value，用 target_net 計算下一個 state 所有動作中最大的 q value (next_q_value)，再計算出 target_q_values。這邊使用 Huber Loss 計算 loss，他在當誤差較大時，轉為線性，對離群值更穩健。

$$L_{DQN}(\theta) = \begin{cases} \frac{1}{2} (r + \gamma \max_{a'} Q(s', a'; \bar{\theta}) - Q(s, a; \theta))^2, & \text{if } |r + \gamma \max_{a'} Q(s', a'; \bar{\theta}) - Q(s, a; \theta)| \leq \delta, \\ \delta |r + \gamma \max_{a'} Q(s', a'; \bar{\theta}) - Q(s, a; \theta)| - \frac{1}{2} \delta^2, & \text{if } |r + \gamma \max_{a'} Q(s', a'; \bar{\theta}) - Q(s, a; \theta)| > \delta. \end{cases} \quad (1)$$

```
1     q_values = self.q_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)
2
3     ##### YOUR CODE HERE (~10 lines) #####
4     # target Q-values for the next states
5     with torch.no_grad():
6         next_q_values = self.target_net(next_states).max(1)[0]
7         target_q_values = rewards + (1 - dones) * self.gamma * next_q_values
8
```

```

9      # Compute the loss
10     # loss = nn.MSELoss()(q_values, target_q_values)
11     loss_fn = nn.SmoothL1Loss()
12     loss = loss_fn(q_values, target_q_values)
13
14     self.optimizer.zero_grad()
15     loss.backward()
16     nn.utils.clip_grad_norm_(self.q_net.parameters(), 1.0)
17     self.optimizer.step()

```

2.1.1 Hyperparameter settings

Parameter	Type	Default Value
Batch Size (--batch-size)	Integer	32
Memory Size (--memory-size)	Integer	200,000
Learning Rate (--lr)	Float	0.0001
Discount Factor (--discount-factor)	Float	0.99
Epsilon Start (--epsilon-start)	Float	1.0
Epsilon Decay (--epsilon-decay)	Float	0.9999
Epsilon Minimum (--epsilon-min)	Float	0.05
Target Update Frequency (--target-update-frequency)	Integer	1,000
Replay Start Size (--replay-start-size)	Integer	5,000
Max Episode Steps (--max-episode-steps)	Integer	10,000
Train Per Step (--train-per-step)	Integer	1

Table 1: Hyperparameter settings for the Task1 implementation.

2.1.2 Result

```

└─> Save to 1 (risking incompatibility):
✓ Saved CartPole episode 18 (Reward: 500.0) → ./cartpole_videos/cartpole_ep18.mp4
IMAGEIO FFMPEG_WRITER WARNING: input image is not divisible by macro_block_size=16, resizing from (600, 400) to (608, 400) to ensure video compatibility with most codecs and players. To prevent resizing, make your input image divisible by the macro_block_size or set the macro_block_size to 1 (risking incompatibility).
✓ Saved CartPole episode 19 (Reward: 500.0) → ./cartpole_videos/cartpole_ep19.mp4
Average reward over 20 episodes: 500.0

```

Figure 1: test result

2.2 Task2 Vanilla DQN on Pong

以 Task1 為基底將 qnet 改成 CNN，其他地方照舊，參數設定把 replay start size 改成 50000 起步，縮小 memory size 成 100K，input state 要經過 AtariPreprocessor 處理。

```

1     elif len(input) == 3: # For Pong
2         print("Using CNN")
3         self.network = nn.Sequential(
4             nn.Conv2d(input[0], 32, kernel_size=8, stride=4),
5             nn.ReLU(),
6             nn.Conv2d(32, 64, kernel_size=4, stride=2),
7             nn.ReLU(),
8             nn.Conv2d(64, 64, kernel_size=3, stride=1),
9             nn.ReLU(),
10            nn.Flatten(),

```

```

11         nn.Linear(64 * 7 * 7, 512),
12         nn.ReLU(),
13         nn.Linear(512, num_actions)
14     )

```

2.3 Task3 Enhanced DQN

2.3.1 Double DQN

Double DQN 是為了解決 DQN 中有 Q 值估計過高的問題 (overestimation)，原始公式 $Q_{\text{target}} = r + \gamma \max_{a'} Q(s', a'; \bar{\theta})$ ，這樣都用同一個 network 去選擇動作和評估會有偏差。因此 Double DQN 的改進是分離選擇和評估的 network， $Q_{\text{target}} = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \bar{\theta})$ ，用 q_net 選擇下一個動作，再用 target_net 估計動作得 Q 值，避免同一個 network 累積偏差的問題。

```

1     with torch.no_grad():
2         next_actions = self.q_net(next_states).argmax(1)
3         next_q_values = self.target_net(next_states).gather(1, next_actions.
4             unsqueeze(1)).squeeze(1)
5         target_q_values = rewards + (1 - dones) * self.gamma * next_q_values

```

2.3.2 Prioritized Experience Replay (PER)

原本的 Experience Replay 是隨機從 buffer 中 uniform sample，PER 先對 sample 計算 Bellman error $\delta = |r + \gamma \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta)|$ ，優先取樣 error 大的 sample，這個 sample 對 network 學習參數比較有價值，下面是實作細節。

利用 sumTree 結構去存 priority 和 sample，加速 PER search 的時間，有了 binary search 的結構可以讓取樣在 $O(\log N)$ 的時間內完成，用 `_retrieve` 找到要更新的 index 去 update priority 和 sample，如果寫入的量 (`write > self.capacity`) 回頭將 write 0 index 覆蓋掉，形成 ring buffer。

```

1     class SumTree:
2         write = 0
3         def __init__(self, capacity):
4             self.capacity = capacity
5             self.tree = np.zeros(2 * capacity - 1) #節點數
6             self.data = np.zeros(capacity, dtype=object) #sample儲存處
7             self.n_entries = 0
8
9             # update to the root node
10            def _propagate(self, idx, change):
11                parent = (idx - 1) // 2
12                self.tree[parent] += change
13                if parent != 0: self._propagate(parent, change)
14
15            # find sample on leaf node
16            def _retrieve(self, idx, s):
17                left = 2 * idx + 1
18                right = left + 1
19                if left >= len(self.tree): return idx
20                if s <= self.tree[left]: return self._retrieve(left, s)
21                else: return self._retrieve(right, s - self.tree[left])
22
23            def total(self):
24                return self.tree[0]
25
26            # store priority and sample
27            def add(self, data, p):
28                idx = self.write + self.capacity - 1
29

```

```

30     self.data[self.write] = data
31     self.update(idx, p)
32
33     self.write += 1
34     if self.write >= self.capacity:
35         self.write = 0
36
37     if self.n_entries < self.capacity:
38         self.n_entries += 1
39
40     # update priority
41     def update(self, idx, p):
42         change = p - self.tree[idx]
43         self.tree[idx] = p
44         self._propagate(idx, change)
45
46     # get priority and sample
47     def get(self, s):
48         idx = self._retrieve(0, s)
49         dataIdx = idx - self.capacity + 1
50         return (idx, self.tree[idx], self.data[dataIdx])

```

PrioritizedReplayBuffer 有兩個參數 α 和 β · α 影響抽樣機率 $P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$ ，當 $\alpha = 0$ 代表所有的樣本抽樣機率相同， $\alpha = 1$ 完全根據 priority 抽樣 · β 控制 IS 校正偏差的程度 $w_i = \left(\frac{1}{N \cdot P(i)}\right)^\beta$ ，當 $\beta = 0$ 時，完全不進行加權，每個樣本的權證和其抽樣機率成反比， $\beta = 1$ 對樣本抽樣偏差校正越強，隨者訓練過程逐漸增大抽樣會越來越 uniform · priority 計算方式要加上一個 e，避免 priority 為 0 的狀況 ·

```

1     class PrioritizedReplayBuffer: # stored as ( s, a, r, s_ ) in SumTree
2         e = 0.01
3         a = 0.6
4         beta = 0.4
5         beta_increment_per_sampling = 0.0005
6
7         def __init__(self, capacity):
8             self.tree = SumTree(capacity)
9             self.capacity = capacity
10            self.epsilon = self.e
11
12            def __len__(self):
13                return self.tree.n_entries
14
15            def _get_priority(self, error):
16                return (abs(error) + self.e) ** self.a
17
18            def add(self, sample, error):
19                p = self._get_priority(error)
20                self.tree.add(sample, p)
21
22            def sample(self, n):
23                batch = []
24                idxs = []
25                segment = self.tree.total() / n
26                priorities = []
27
28                self.beta = np.min([1., self.beta + self.beta_increment_per_sampling])
29
30                for i in range(n):
31                    a = segment * i

```

```

32         b = segment * (i + 1)
33
34         s = random.uniform(a, b)
35         (idx, p, data) = self.tree.get(s)
36         priorities.append(p)
37         batch.append(data)
38         idxs.append(idx)
39
40         sampling_probabilities = priorities / self.tree.total()
41         is_weight = np.power(self.tree.n_entries * sampling_probabilities, -self.beta)
42         is_weight /= is_weight.max()
43
44         return batch, idxs, is_weight
45
46     def update(self, idx, error):
47         p = self._get_priority(error)
48         self.tree.update(idx, p)

```

在訓練過程中要將權重加入計算 loss，計算完的 td_error 丟入 update 重新更新 priority。

```

1         loss_fn = nn.SmoothL1Loss(reduction='none')
2         losses = loss_fn(q_values, target_q_values)
3
4         # Normalize weights
5         weights = torch.tensor(weights, dtype=torch.float32, device=self.device)
6         loss = (losses * weights.to(self.device)).mean()
7
8         td_errors = torch.abs(target_q_values - q_values).detach().cpu().numpy()
9         for i, idx in enumerate(indices):
10             self.memory.update(idx, td_errors[i])

```

2.3.3 Multi-Step Return

Multi-step return 做法是考量未來多步的累積 reward $R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_{a'} Q(s_{t+n}, a')$ ，更快地接收到 reward 的回饋，加速學習過程。

```

1         R = sum([self.gamma**i * self.n_step_buffer[i][2] for i in range(self.n_step)])
2         s0, a0, _, _, _ = self.n_step_buffer[0]
3         _, _, _, s_n, done_n = self.n_step_buffer[-1]
4         td_error = abs(R + (1-done_n) * (self.gamma ** self.n_step) * next_q_val - q_val)
5         self.memory.add((s0, a0, R, s_n, done_n), td_error)

```

2.3.4 Weight & Bias

使用 wandb 紀錄實驗數據，他有很好的視覺化系統，可以呈現不同參數設定下的數據變化，選擇不同的 x,y 軸和比例。

3 Analysis and discussions

3.1 Training curves

3.1.1 task1

可以觀察到大概在 env step 250K 就可以收斂到很好的 performance 了。

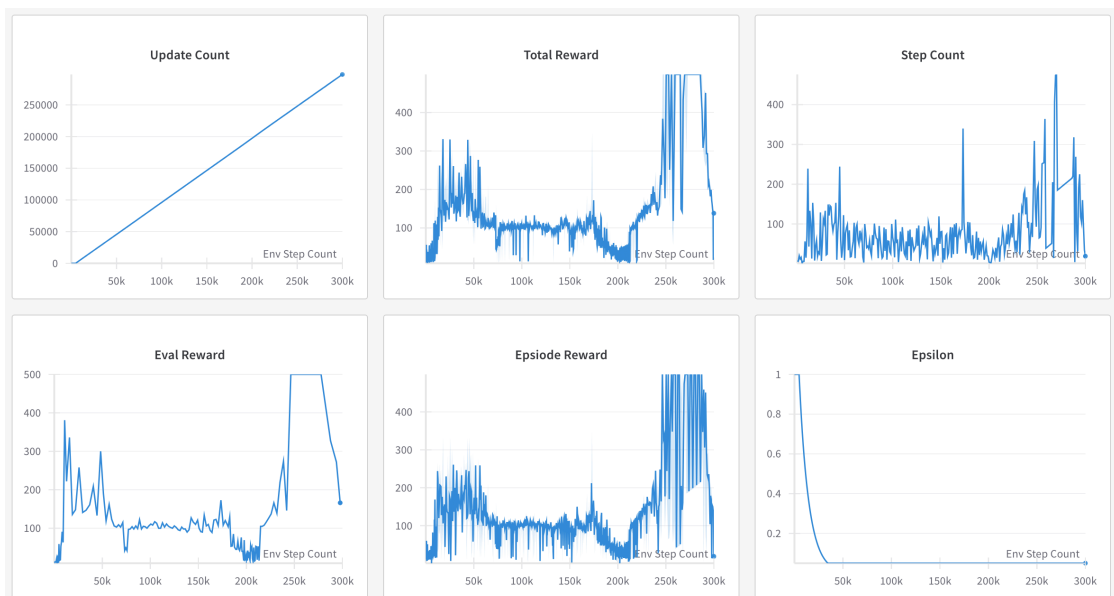


Figure 2: CartPole Result

3.1.2 task2

大概在 2M 左右會收斂，依據不同的 random seed 平均 reward 可以到 19 上下

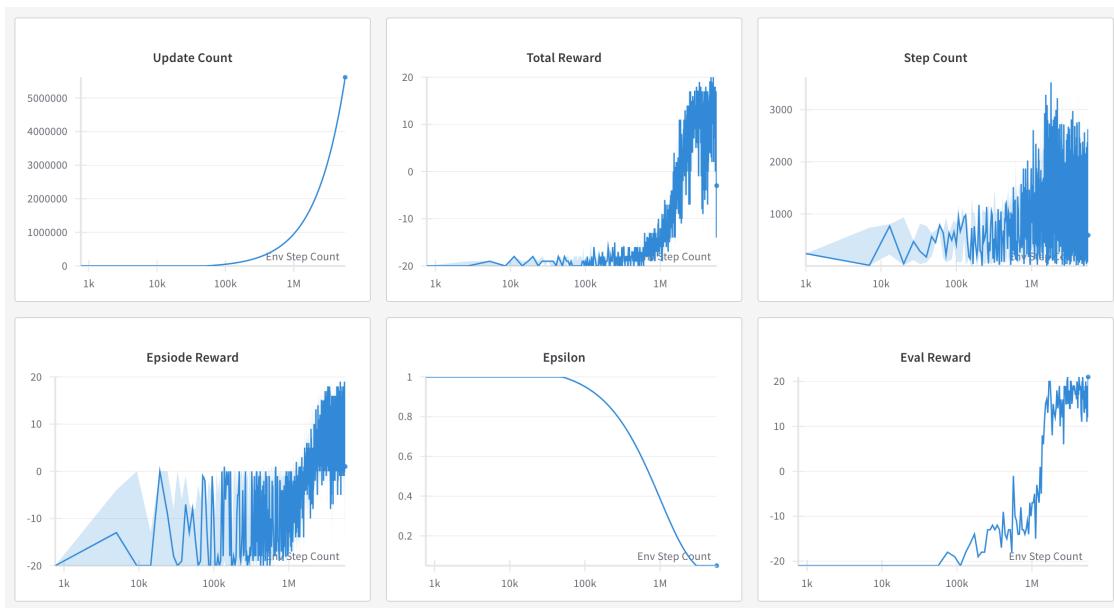


Figure 3: Pong Result

3.1.3 task3

增加了 enhanced 的功能之後，可以在前期快速上升 reward 確實有顯著的效果，但後期需要收斂的 step 數仍然與 task2 差不多。加上學習速度太快的緣故，後期 reward 震盪幅度大，收斂效果不穩定。下圖是不同的超參數設定下的結果。大約在 1M 2M 之間可以 reach 19。

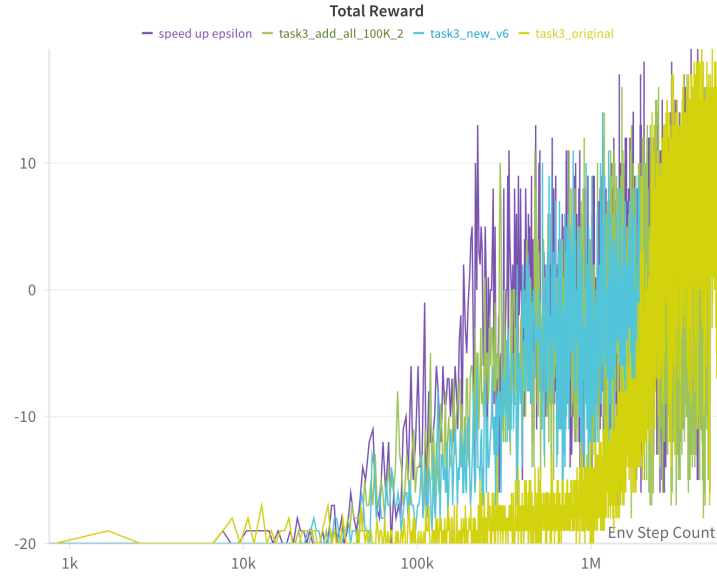


Figure 4: total reward

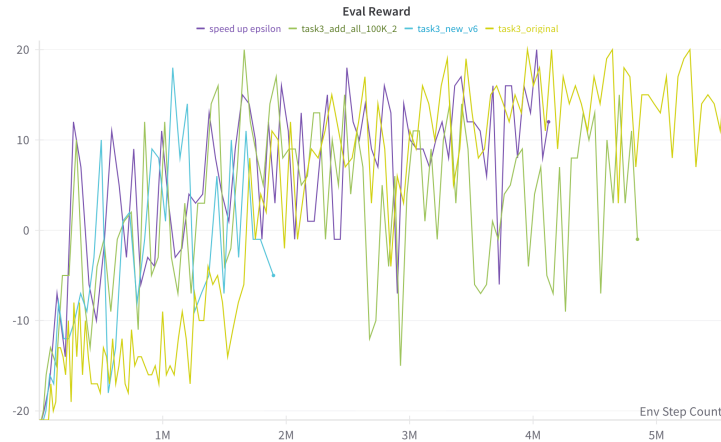


Figure 5: eval reward

3.2 Ablation study

3.2.1 DDQN Overestimation

從下圖看出，在加了 double DQN 之後有效降的 Q value 估計過高的問題

3.2.2 DDQN、PER、Multi-Step Return

在使用相同參數設定下，比較 total reward(Figure 8) 和 20 次的 eval reward(Table 2)。這邊為了增加 sample efficiency, 將 epsilon decay 速度調高，加速 network greedy 的速度，可以發現他確實能有效在前期就有不錯的平均 reward，但後期上升較困難，會有 overfitting 的風險，加上 DDQN 和 PER 對整體 reward 收斂速度加快的影響程度不大，需要搭配其他有效的參數調整，可以加快前期 evaluate reward 的成效。

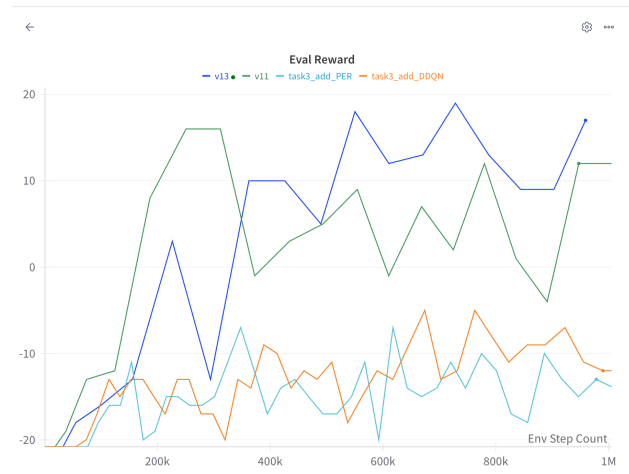
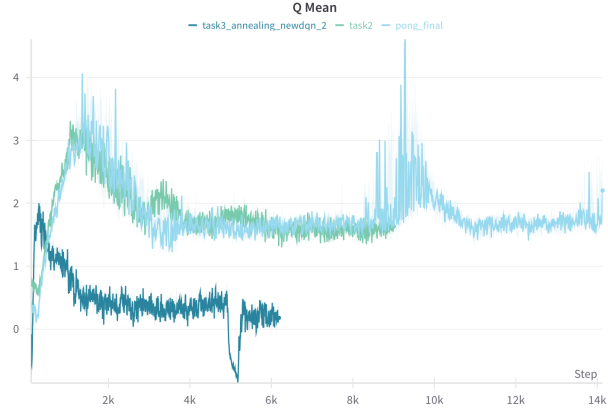


Figure 6: Evaluation curves, v11 v13 是 epsilon 加速 decay 的版本

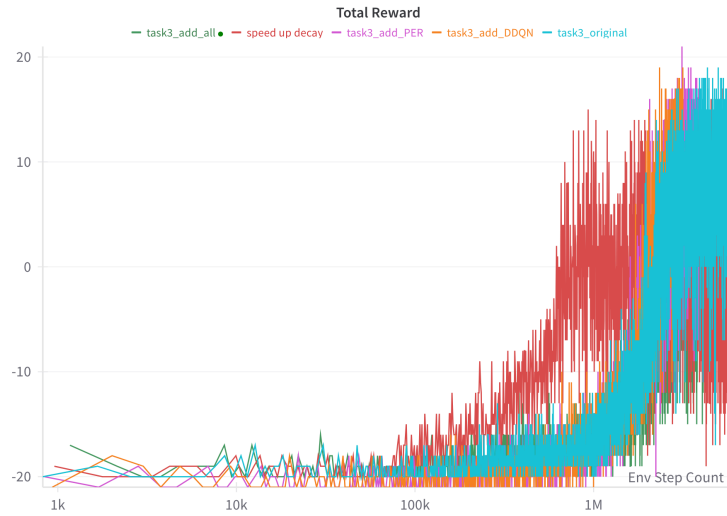


Figure 7: Total reward curves

	original	add_DDQN	add_PER	add_all(with speed up epsilon-decay)
200K	-14.15	-12.10	-13.15	3.65
400K	-11.90	-14.55	-15.50	8.25
600K	-12.70	-11.60	-12.95	11.80
800K	-12.05	-10.30	-13.60	12.30
1M	-8.85	-6.05	-9.8	12.35

Table 2: 20 average reward for each step

3.3 Additional training strategies

3.3.1 Test different train per step on Cartpole

從實驗結果可以得知 train per step 在 cartpole 任務中越高越好，可以更快訓練完成，但這樣的訓練方法容易在 evaluate 的時候有震盪，為了減少震盪加入了 soft update，讓 network 每次只更新一點點，由結果可知，後期收斂之後的震盪擺幅沒有那麼大。

```

1  def soft_update(self, tau=0.005):
2      for target_param, param in zip(self.target_net.parameters(), self.q_net.
3          parameters()):
4          target_param.data.copy_(tau * param.data + (1.0 - tau) * target_param.data
5
6  if self.train_count % self.target_update_frequency == 0:
7      self.soft_update(tau=0.005)

```

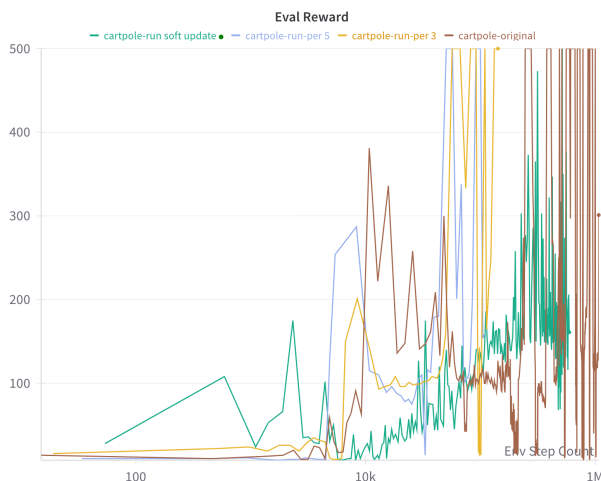


Figure 8: compare result

3.3.2 Test different epsilon decay methods

將 epsilon 的 decay 方式改成 cycling，因為觀察到如果下降速度過快，後期隨機探索太少的話 reward 會卡住震盪不容易上升，在前 500k 改成 cycling 避免這個狀況。

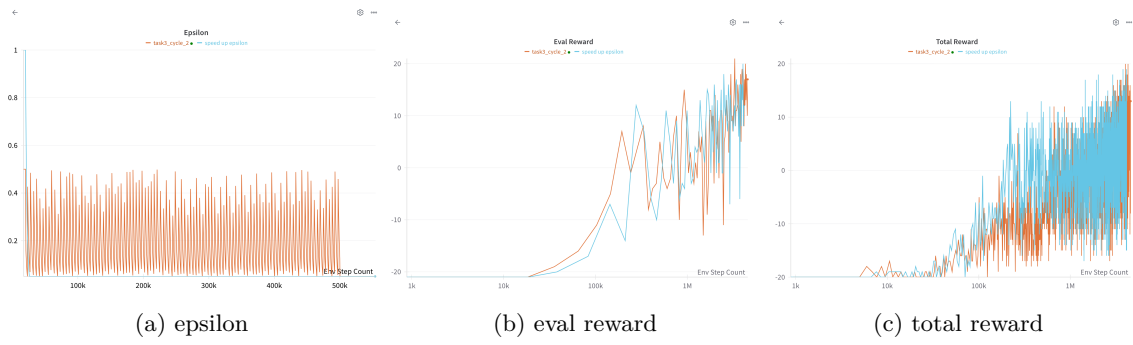


Figure 9: cycle decay